

Blocco 12 - Capstone architettura DBMS e dashboard

Dalla specifica al sistema containerizzato completo

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Capstone

L'obiettivo finale non è una singola query, ma un sistema piccolo, spiegabile e replicabile.

Obiettivo del blocco

Problema

Integrare tutto: Docker Compose, collector simulato, PostgreSQL, schema relazionale, viste SQL, query Metabase, dashboard e controlli.

Idea guida

Una soluzione professionale descrive architettura, dati, query, verifiche e limiti.

Cosa deve essere consegnato

- ▶ una descrizione dell'architettura;
- ▶ il comando per avviare i container;
- ▶ lo schema dati e il ruolo delle tabelle;
- ▶ almeno cinque query per dashboard;
- ▶ una dashboard Metabase costruita con quelle query;
- ▶ controlli di qualità e performance;
- ▶ una breve spiegazione dei limiti.

Elemento	Descrizione
Collector	genera ticket simulati e li invia al DBMS
PostgreSQL	conserva raw, curated, eventi e viste dashboard
Metabase	interroga PostgreSQL e presenta card e dashboard
SQL	definisce schema, trasformazioni, metriche e controlli
Docker Compose	rende l'ambiente riproducibile

Da richiesta informale a specifica

Richiesta iniziale

“Voglio vedere come stanno andando i ticket e capire dove intervenire.”

Questa frase non è ancora una specifica: mancano popolazione, metriche, granularità, periodo, filtri, visualizzazioni e controlli.

- ▶ popolazione: ticket nello schema `ticketing`;
- ▶ metrica 1: ticket aperti, chiusi e violazioni SLA;
- ▶ metrica 2: trend giornaliero e backlog;
- ▶ metrica 3: ranking categorie e tempi medi;
- ▶ filtri: data apertura, priorità, regione, categoria;
- ▶ output: dashboard Metabase con query SQL versionate.

2. Avvio del sistema

La riproducibilità parte dai comandi, non dalle slide.

Comandi di avvio

```
# Dalla root della repository studenti
docker compose -f docker-compose.ticketing.yml up -d

# Controllo container
docker compose -f docker-compose.ticketing.yml ps

# Log del collector simulato
docker logs -f rdsql-ticket-collector
```

Servizio	Accesso
PostgreSQL Database	host localhost, porta 5433
Utente/password	training
Metabase	training/training
	http://localhost:3000

Connessione psql

```
docker exec -it rdsq1-ticket-postgres \  
    psql -U training -d training
```

```
-- dentro psql
```

```
SET search_path TO ticketing;
```

```
SELECT count(*) FROM support_tickets;
```

Connessione Metabase a PostgreSQL

- ▶ aprire `http://localhost:3000`;
- ▶ creare l'account iniziale;
- ▶ aggiungere database PostgreSQL;
- ▶ host: `postgres` se Metabase gira nel Compose;
- ▶ porta: `5432` dentro la rete Docker;
- ▶ database, utente e password: `training`.

3. Query della dashboard

Ogni card deve poter essere spiegata e verificata.

Set minimo di card

- ① KPI complessivi;
- ② serie giornaliera aperti/risolti/backlog;
- ③ distribuzione priorità-stato;
- ④ tempo medio di risoluzione per priorità e regione;
- ⑤ ranking categorie;
- ⑥ tabella ticket aperti per drill-down.

Card 1: KPI

```
SELECT count(*) AS total_tickets,  
       count(*) FILTER (WHERE closed_at IS NULL) AS open_tickets,  
       count(*) FILTER (WHERE closed_at IS NOT NULL) AS closed_tickets,  
       count(*) FILTER (WHERE sla_breached) AS sla_breached_tickets  
FROM ticketing.dashboard_ticket_base;
```

Card 2: trend giornaliero

```
SELECT day,  
       opened_tickets,  
       resolved_tickets,  
       backlog_delta  
FROM ticketing.dashboard_daily_flow  
ORDER BY day;
```

Card 3: ranking categorie

```
SELECT category,  
       count(*) AS tickets,  
       rank() OVER (ORDER BY count(*) DESC, category) AS category_rank  
FROM ticketing.dashboard_ticket_base  
GROUP BY category  
ORDER BY category_rank, category;
```

Card 4: ticket aperti

```
SELECT ticket_id, source_code, external_ticket_id,  
       opened_at, priority, category, region,  
       customer_segment, channel, subject,  
       sla_due_at, sla_breached  
FROM ticketing.dashboard_ticket_base  
WHERE closed_at IS NULL  
ORDER BY priority DESC, opened_at;
```

Costruzione dashboard in Metabase

- ▶ creare ogni query come “Native query”;
- ▶ salvare la query con nome descrittivo;
- ▶ scegliere visualizzazione coerente con la granularità;
- ▶ aggiungere le domande alla dashboard;
- ▶ configurare filtri comuni solo dove hanno lo stesso significato.

4. Verifica

Un sistema è credibile se può essere controllato.

- ▶ raw event e ticket curati sono allineati?
- ▶ i ticket aperti nella dashboard coincidono con la tabella base?
- ▶ i conteggi aggregati tornano al totale?
- ▶ le violazioni SLA sono coerenti con `sla_due_at`?
- ▶ il collector sta aggiungendo nuove righe?

Controllo raw-curated

```
SELECT (SELECT count(*) FROM ticketing.support_tickets_raw) AS raw_events,  
(SELECT count(*) FROM ticketing.support_tickets) AS curated_tickets,  
(SELECT count(*)  
  FROM ticketing.support_tickets_raw r  
  LEFT JOIN ticketing.support_tickets t  
    ON t.external_ticket_id = r.external_ticket_id  
 WHERE t.ticket_id IS NULL) AS raw_without_curated_ticket;
```

Controllo performance

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, opened_at, priority, region, status
FROM ticketing.support_tickets
WHERE status IN ('open', 'assigned', 'waiting_customer')
ORDER BY opened_at DESC, ticket_id DESC;
```

5. Indici e trade-off

Un indice non è un trucco per rendere tutto più veloce: è una struttura dati che accelera alcune letture e rende più costose alcune scritture.

Perché aumentare il carico

Problema didattico

Con poche decine di ticket PostgreSQL può leggere tutta la tabella in memoria: l'indice sembra inutile perché il costo di attraversarlo è simile al costo di leggere tutto.

Idea

Generiamo migliaia di ticket distribuiti nel tempo. A quel punto una query dashboard che chiede “ultimi ticket aperti” non deve più guardare tutto: deve trovare rapidamente una piccola parte ordinata.

Generare eventi ticketing nel tempo

```
# 80.000 ticket sintetici distribuiti su 150 giorni
docker exec -i rdsq1-ticket-postgres \
  psql -U training -d training \
  -v load_size=80000 \
  -f /sql/ticket_load_generate.sql
```

Lo script genera ticket, raw event ed eventi di lifecycle: created, assigned, comment, status change, resolved o closed.

Cosa osservare dopo il carico

Osservazione	Perché conta
numero di ticket	aumenta il costo di una scansione completa
numero di eventi	mostra il costo della storia operativa
distribuzione temporale	rende realistiche query su finestre recenti
ticket aperti vs chiusi	rende utile un indice parziale
filtri regione/categoria	rende utile un indice composito

Query target prima degli indici

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, opened_at, priority, region, status
FROM ticketing.support_tickets
WHERE status IN ('open', 'assigned', 'waiting_customer')
      AND opened_at >= TIMESTAMP '2026-03-01'
ORDER BY opened_at DESC, ticket_id DESC
LIMIT 50;
```

Indice parziale per ticket aperti

```
CREATE INDEX idx_support_tickets_open_dashboard  
ON ticketing.support_tickets (opened_at DESC, ticket_id DESC)  
INCLUDE (priority, region, status, category, sla_due_at)  
WHERE status IN ('open', 'assigned', 'waiting_customer');
```

L'indice contiene solo i ticket non chiusi: è più piccolo di un indice su tutti i ticket e risponde bene al drill-down operativo.

Indice composito per filtri dashboard

```
CREATE INDEX idx_support_tickets_region_category_date  
ON ticketing.support_tickets (region, category, opened_at DESC)  
INCLUDE (status, priority, closed_at, sla_due_at);
```

Questo indice aiuta le card filtrate per area, categoria e periodo. Non aiuta invece query che filtrano con colonne diverse o con ordinamenti incompatibili.

Trade-off degli indici

Beneficio	Costo	Domanda da fare
meno letture per query selettive	più spazio su disco	questa query è frequente?
ordinamenti già disponibili	INSERT/UPDATE più lenti	quanto scriviamo ogni minuto?
dashboard più reattiva	più manutenzione schema	chi mantiene gli indici?
meno carico CPU/I/O	rischio indici inutilizzati	il piano li usa davvero?

Script dell'esperimento completo

```
docker exec -i rdsq1-ticket-postgres \  
    psql -U training -d training \  
    -f /sql/ticket_index_tradeoff.sql
```

Lo script misura: piano prima degli indici, costo di scrittura prima degli indici, creazione degli indici, dimensione degli indici, piano dopo gli indici e costo di scrittura dopo gli indici.

- ▶ il collector è simulato, non legge una vera API;
- ▶ il carico sintetico rende visibile il fenomeno, ma non sostituisce dati di produzione;
- ▶ gli indici scelti sono coerenti con alcune query dashboard, non con tutte;
- ▶ Metabase usa una configurazione locale semplificata;
- ▶ non sono trattati autenticazione, backup e sicurezza di produzione;
- ▶ l'obiettivo è didattico: rendere replicabile il ragionamento architetturale.

6. Presentazione finale

Ogni gruppo deve difendere sia il modello sia le query e deve discutere i compromessi tecnici introdotti.

Struttura della presentazione

- ➊ architettura e responsabilità dei container;
- ➋ modello dati e scelte raw/curated;
- ➌ viste SQL esposte alla dashboard;
- ➍ card Metabase e significato delle metriche;
- ➎ controlli dati, performance e indici;
- ➏ limiti e possibili estensioni.

- ▶ chiarezza della specifica;
- ▶ coerenza tra architettura, schema e query;
- ▶ correttezza SQL;
- ▶ qualità delle spiegazioni;
- ▶ presenza di controlli;
- ▶ capacità di discutere trade-off tra letture, scritture, spazio e manutenzione.

Estensioni possibili

- ▶ aggiungere un secondo DB per separare operational e analytics;
- ▶ introdurre una tabella eventi più ricca;
- ▶ aggiungere refresh pianificato di viste materializzate;
- ▶ creare filtri Metabase per priorità, regione e periodo;
- ▶ simulare errori raw non caricabili nel curated.

Organizzazione dei 90 minuti

- ▶ 20 min avvio stack e controllo schema;
- ▶ 20 min query e card Metabase;
- ▶ 20 min carico sintetico ed esperimento sugli indici;
- ▶ 15 min dashboard, controlli e filtri;
- ▶ 10 min presentazione sintetica.

- ▶ il capstone collega SQL, DBMS, container e dashboard;
- ▶ una dashboard affidabile nasce da un modello dati affidabile;
- ▶ gli indici sono una scelta architetturale, non un automatismo;
- ▶ la consegna finale deve essere replicabile, spiegabile e verificabile.